Apache NiFi Assignment: Real-Time Data Ingestion & Processing

Objective

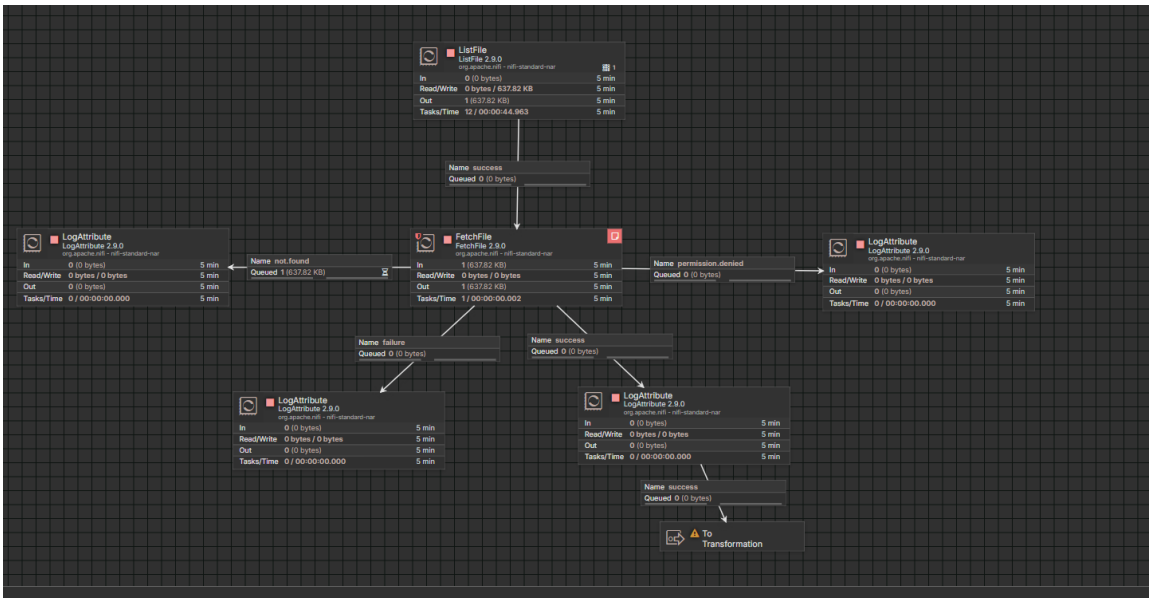
Design and implement a complete data ingestion pipeline using Apache NiFi that simulates real-time data, processes it, and stores it in a distributed system.

1. Real-Time Data Simulation

- **Real-Time Data Production:** Python scripts act as continuous producers, generating semi-structured data in both **JSON** and **SQL** formats.
- **Dual-Source Delivery:** Data is delivered via a **PostgreSQL** database and a local directory mapped to a **Docker volume** for immediate NiFi ingestion.
- **Intentional "Dirty" Data:** To simulate real-world challenges, the scripts inject flaws like **missing values (nulls)**, **inconsistent date formats**, and **mixed data types**.
- **Robust Testing:** By using a constrained ID range, the simulation forces **duplicate records**, creating a rigorous environment to test NiFi's cleansing and de-duplication logic.
- **Streaming Stress Test:** The continuous load provides a realistic scenario for evaluating pipeline stability and parsing accuracy under pressure.

2. File-Based Ingestion using NiFi

I implemented a robust data ingestion pipeline in Apache NiFi using the best-practice ListFile/FetchFile pattern to consume the generated streaming data. The workflow begins with a ListFile processor that identifies new files in the target directory without maintaining a permanent hold on them, ensuring high performance and scalability. These metadata-only flowfiles are then passed to the FetchFile processor, which pulls the actual content into the NiFi ecosystem. To ensure production-grade reliability and zero data loss, I configured comprehensive error-handling routes, directing flowfiles to specific LogAttribute processors for scenarios such as not.found, permission.denied, and general failure. This setup guarantees that every generated order is accounted for, while successfully ingested data is seamlessly routed to the next phase for transformation and cleansing.



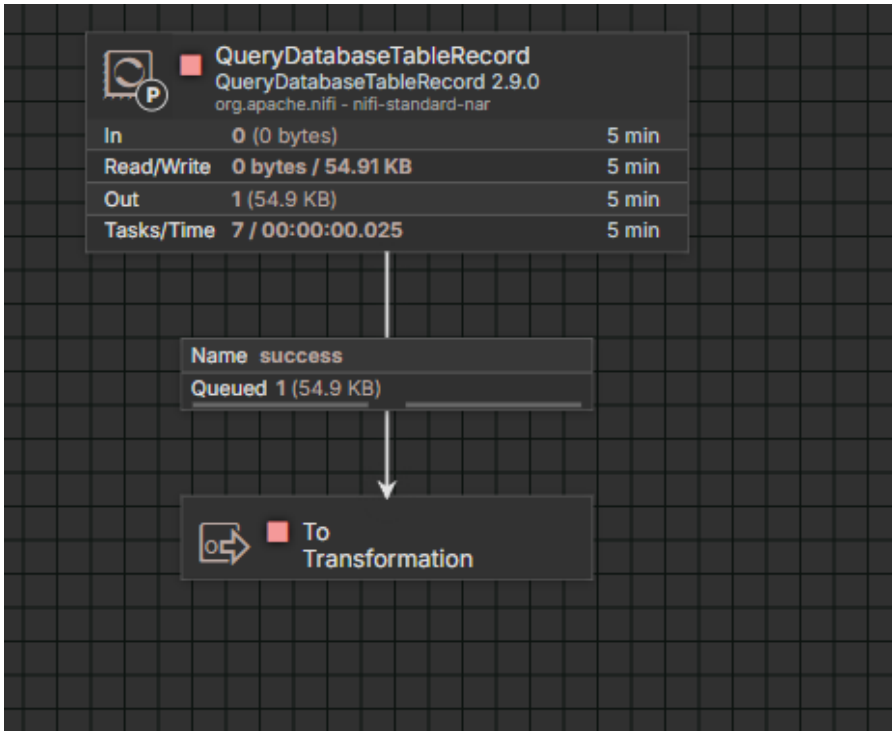
3. Database Ingestion

Real-Time Integration: Connected to PostgreSQL via QueryDatabaseTableRecord for live data streaming.

Incremental Extraction: Tracks a "Maximum-value Column" (e.g., ID) to fetch only new records, avoiding heavy full table scans.

Automated Conversion: Instantly converts SQL rows into JSON/Avro formats during ingestion to maintain structure.

Pipeline Efficiency: Ensures high-speed, data-consistent delivery to the downstream transformation group.



4. Data Transformations

Advanced Processing and Data Refinement Phase:

A. Content Aggregation (MergeContent):

I integrated the MergeContent processor as a pivotal step to aggregate streaming data from disparate sources (both Python-generated files and PostgreSQL records). This feature bundles small, individual records into larger data batches before downstream processing. This significantly enhances system throughput, reduces resource overhead, and ensures that data is handled as cohesive sets rather than fragmented files.

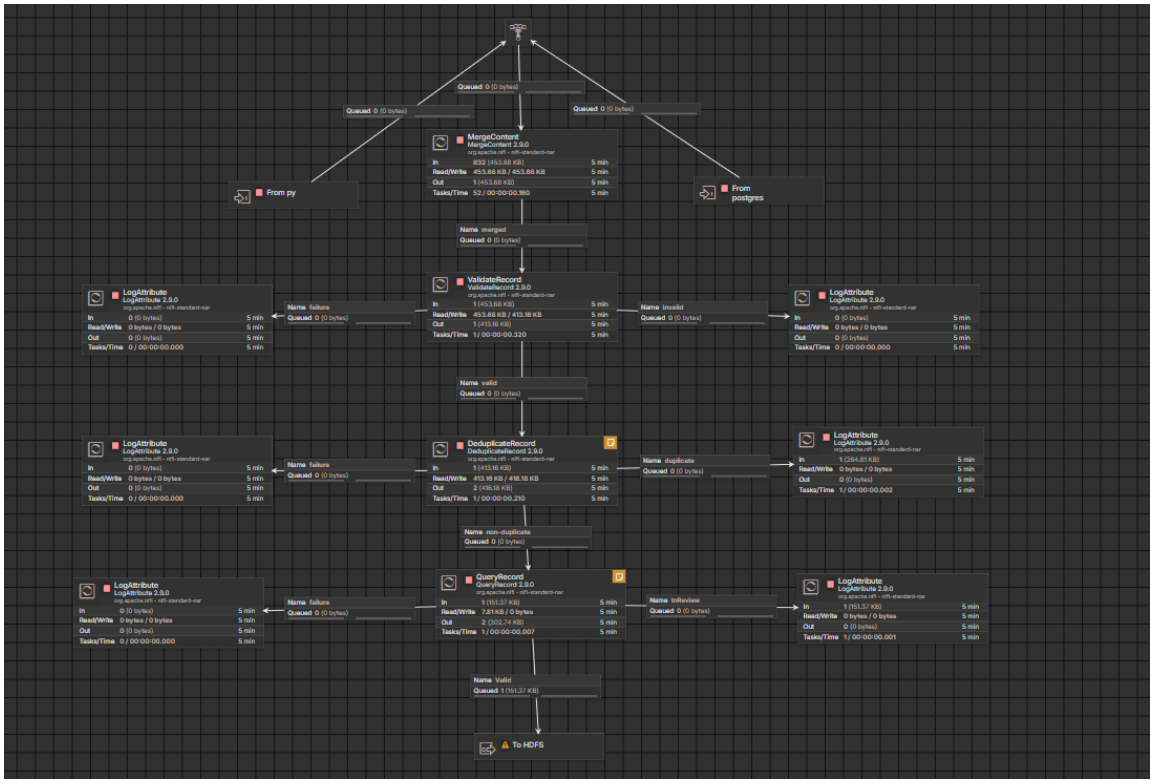
B. Structural Validation (ValidateRecord):

I designed a flexible schema capable of accepting diverse data types, including both strings and decimals. This prevents pipeline failures when encountering mixed-format financial values. Any record violating this schema is automatically routed to an "Invalid" path, ensuring the structural integrity of the digital pipeline and preventing system crashes.

C. Real-time De-duplication (DeduplicateRecord):

I implemented an intelligent mechanism to detect duplicate identifiers within each data batch in real-time. This logic isolates redundant entries and redirects them to a "Duplicate" path, maintaining the accuracy and uniqueness of the final dataset.

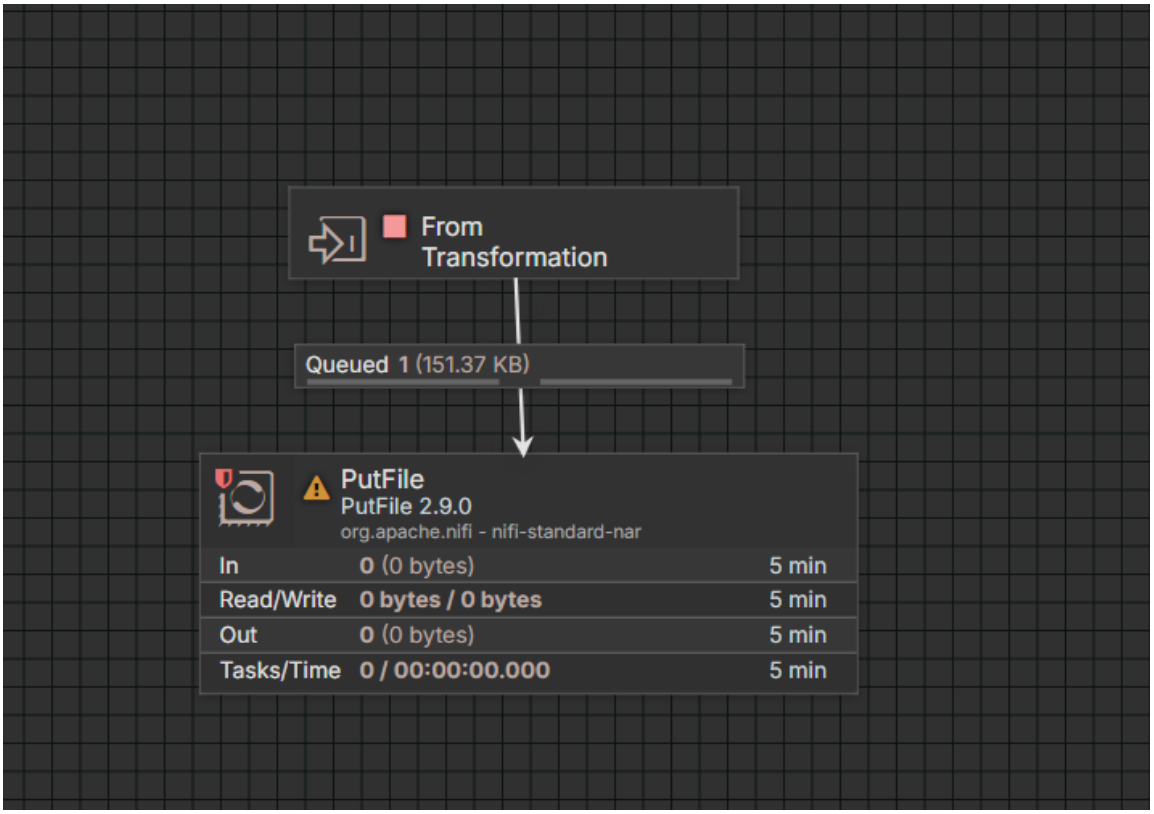
D. Advanced Transformation and Smart Querying (QueryRecord)



5. Load to HDFS

Due to local environment limitations, PutFile was used to simulate HDFS storage

- Final Storage: Refined data is routed to the PutFile processor for final persistence.
- Local Landing Zone: Records are stored in a designated local directory as a secure "landing zone."
- Reliability: Using PutFile ensures data is safely saved and accessible even without an HDFS connection.
- Pipeline Completion: This stage marks the successful transition from raw streaming data to clean, stored output.



6. Apply NiFi Best Practices

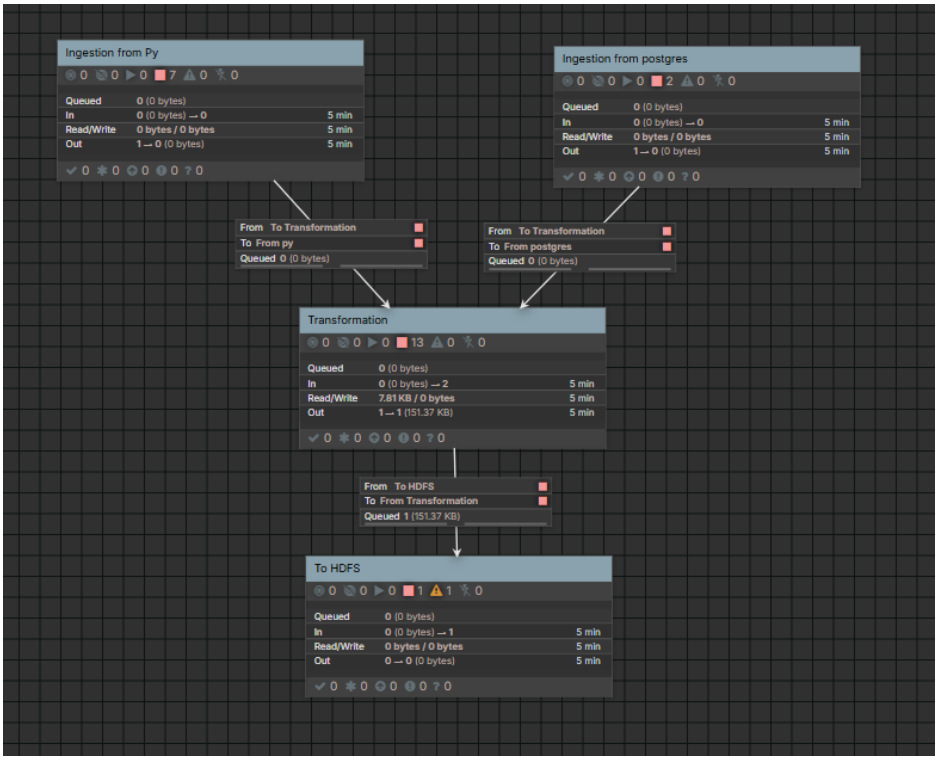
- Back Pressure: I configured data limits on connections between components to prevent congestion and protect system memory from exhaustion during high-volume data surges.
- Yield Duration: I set a 5-second yield duration for processors connected to external systems (PostgreSQL and local storage). This allows the system to pause temporarily upon failure, reducing unnecessary stress on the infrastructure.
- Retry & Penalization: I enabled the penalization feature to ensure failed files are not immediately reprocessed. This gives the system time to stabilize before attempting a retry, which I applied to the data loading stages.
- Prioritizes: I implemented a First-In-First-Out (FIFO) priority at the data merging point within the processing section to maintain the chronological order of incoming records.
- Relationship Handling: I avoided using "Auto-Terminate" on critical relationships. Instead, I routed all failure scenarios to LogAttribute processors to ensure zero data loss and to simplify error tracking and debugging.

7. Additional Ingestion Techniques

- Database (PostgreSQL): Used QueryDatabaseTableRecord for direct connection and automatic JSON conversion of live table data.
- Python Integration: Implemented folder monitoring to ingest, process, and delete files generated by external scripts, proving cross-tool compatibility.

Deliverables

1. NiFi Flow (template or screenshots**) Workflow screenshots are included**
2. Simulation Script: The Python script file used for real-time data generation is attached
3. Architecture Diagram: The process group is provided
4. Documentation explaining design decisions
5. Sample data before and after transformation



Data Before and After processing

	id [PK] integer	order_id integer	customer_name text	product text	price text	quantity integer	city text	order_date text
1	1	1030	[null]	Headphon...	300	2	Taiz	2026-05-07 19:07:...
2	2	1030	[null]	Headphon...	[null]	1	[null]	2026-05-07 19:07:...
3	3	1065	Ali Ahmed	Headphon...	100	3	Sana'a	2026-05-07 19:07:...
4	4	1053	Omar Hassan	Laptop	200	2	[null]	2026-05-07 19:07:...
5	5	1020	[null]	Laptop	100	2	Aden	07/05/2026


```
[
  {
    "id": 3,
    "order_id": 1065,
    "customer_name": "Ali Ahmed",
    "product": "Headphones",
    "price": 100.0,
    "quantity": 3,
    "city": "Sana'a",
    "order_date": "2026-05-07"
  },
  {
    "id": 6,
    "order_id": 1087,
    "customer_name": "Ali Ahmed",
    "product": "Headphones",
    "price": 0.0,
    "quantity": 3,
    "city": "Aden",
    "order_date": "2026-05-07"
  },
```